



AI-Powered Billing Management

Backend Internship — Complete Project Report

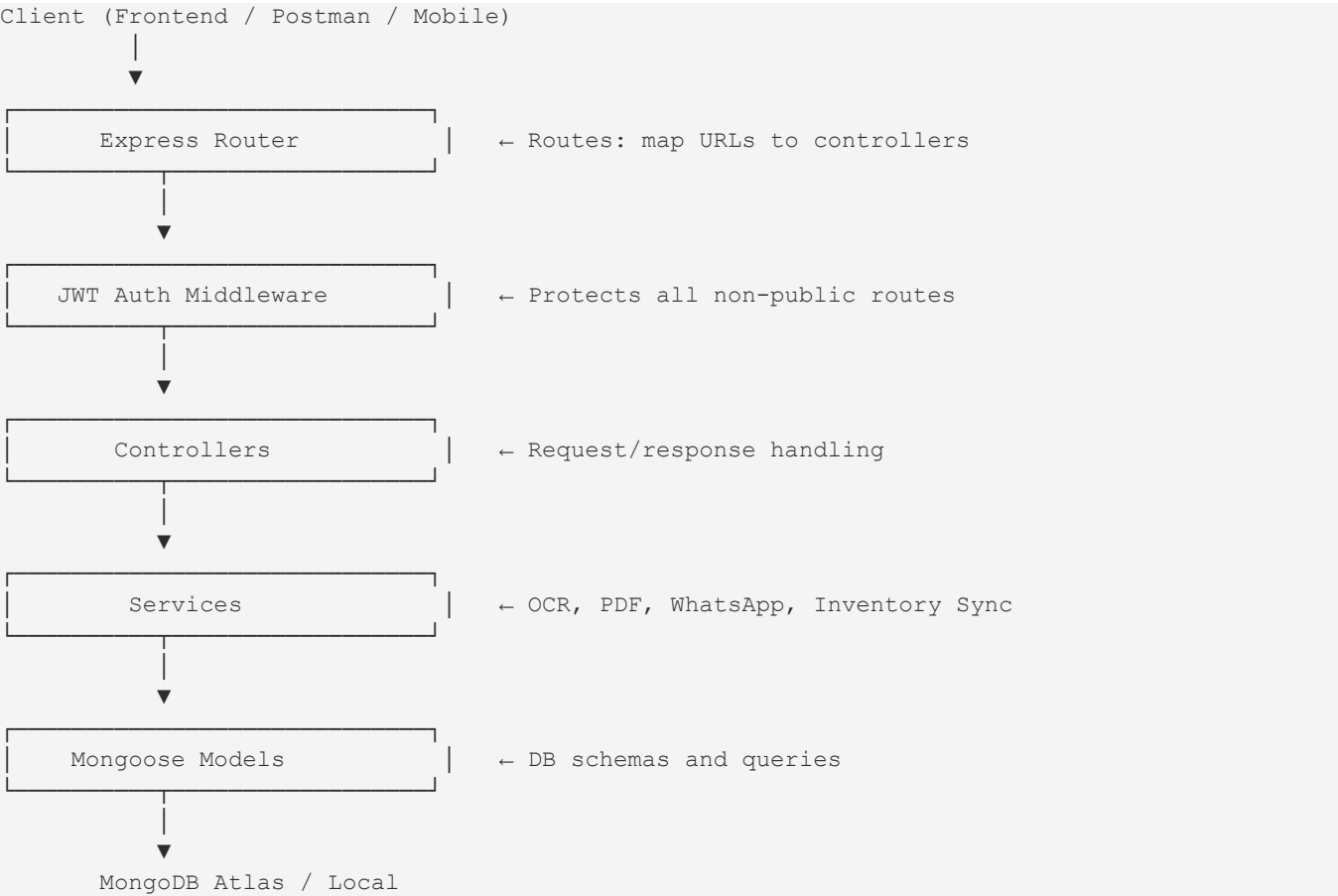
Contents

1. Backend Architecture
 2. API Endpoint Reference
 3. Code Files Overview
 4. Desktop Setup Guide
 5. Special / Innovation Features
 6. Day-by-Day Work Log (11–26 May 2026)
-

1. Backend Architecture

The backend follows a Layered MVC Architecture built on Node.js and Express.js. Each layer has a single responsibility, making the codebase maintainable and testable.

1.1 Layer Diagram



1.2 Technology Stack

Layer	Technology	Purpose
Runtime	Node.js v18+	JavaScript server environment
Framework	Express.js 4.x	HTTP routing and middleware
Database	MongoDB + Mongoose	NoSQL data storage
Authentication	JWT + bcryptjs	Stateless auth, password hashing
OCR Engine	Tesseract.js	Optical character recognition
Image Processing	Sharp	Image enhancement for OCR accuracy
PDF Generation	PDFKit	Professional downloadable invoices
File Upload	Multer	Multipart form / image handling

WhatsApp	Twilio API	Receipt delivery to customer [Innovation]
Testing	Jest + Supertest	Unit and integration tests

1.3 Folder Structure

```

billing-backend/
├── server.js                # Entry point - Express app init
├── package.json            # Dependencies and scripts
├── .env.example            # Environment variable template
├── .gitignore
├── config/
│   ├── db.js              # MongoDB connection
│   └── multer.js          # File upload config
├── models/
│   ├── User.js            # Shop owner schema
│   ├── Product.js        # Inventory schema + expiry tracker
│   └── Bill.js            # Invoice schema + auto bill number
├── routes/                # URL → controller mapping (6 files)
├── controllers/           # Request handlers (6 files)
├── middleware/
│   ├── auth.middleware.js # JWT verification
│   └── error.middleware.js # Global error handler
├── services/
│   ├── ocr.service.js
│   ├── imageProcessor.service.js
│   ├── inventorySync.service.js
│   ├── pdfGenerator.service.js
│   └── whatsapp.service.js # [Innovation]
├── utils/
│   ├── responseHelper.js
│   ├── tokenHelper.js
│   ├── dateHelper.js
│   └── ocrParser.js
├── innovation/            # Special feature demos + docs
├── tests/                 # Jest test files
├── docs/                  # All project documentation
└── uploads/               # Bill images (gitignored)

```

1.4 OCR Bill Scanning — Data Flow

1. User uploads bill image → POST /api/ocr/scan
2. Multer saves image to /uploads/
3. Sharp preprocesses: grayscale → normalize → sharpen → binarize
4. Tesseract.js extracts raw text
5. ocrParser.js parses: shopName, date, line items[]
6. Frontend shows parsed data for user review
7. User confirms → POST /api/bills (creates draft)
8. PUT /api/bills/:id/finalize

- |— inventorySync.service.js deducts stock quantities
 - |— Bill status → "finalized"
 - |— (Optional) WhatsApp receipt sent to customer
-

2. API Endpoint Reference

Base URL: `http://localhost:5000/api` | Protected routes require: Authorization: Bearer <token>

2.1 Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/auth/register	✗ Public	Register new shop owner
POST	/auth/login	✗ Public	Login and receive JWT token
GET	/auth/me	☑ Required	Get current logged-in user profile
PUT	/auth/darkmode	☑ Required	Toggle dark mode preference [Innovation]

2.2 Product / Inventory Endpoints

Method	Endpoint	Auth	Description
GET	/products	☑	Get all products (search, category, lowStock, expired filters)
POST	/products	☑	Add a new product to inventory
GET	/products/:id	☑	Get single product by ID
PUT	/products/:id	☑	Update product details or quantity
DELETE	/products/:id	☑	Soft-delete product (preserves bill history)
GET	/products/categories	☑	Get list of all unique categories

2.3 Billing / Invoice Endpoints

Method	Endpoint	Auth	Description
POST	/bills	☑	Create a new draft bill with items
GET	/bills	☑	Get all bills (status, date-range, pagination filters)
GET	/bills/:id	☑	Get single bill by ID
PUT	/bills/:id/finalize	☑	Finalize bill — auto-deducts inventory stock
PUT	/bills/:id/cancel	☑	Cancel a draft bill

2.4 OCR, PDF & Dashboard Endpoints

Method	Endpoint	Auth	Description
POST	/ocr/scan	☒	Upload bill image → returns parsed shop name, date, items
GET	/pdf/invoice/:id	☒	Download professional PDF invoice for a bill
GET	/dashboard/summary	☒	Today revenue, totals, low-stock & expiry counts
GET	/dashboard/revenue	☒	Daily revenue data for last N days (bar chart)
GET	/dashboard/top-products	☒	Top-selling products by quantity

2.5 Sample Request & Response

POST /api/bills — Create Bill

Request Body (JSON):

```
{
  "customerName": "John Doe",
  "customerPhone": "9876543210",
  "paymentMethod": "upi",
  "taxRate": 5,
  "discount": 20,
  "items": [
    { "product": "64a1...", "productName": "Rice", "quantity": 2, "unitPrice": 250 },
    { "productName": "Sugar", "quantity": 1, "unitPrice": 45 }
  ]
}
```

Response 201:

```
{
  "success": true,
  "message": "Bill created",
  "data": {
    "billNumber": "BILL-1717000000-0001",
    "subtotal": 545,
    "taxAmount": 27.25,
    "totalAmount": 552.25,
    "status": "draft"
  }
}
```

3. Code Files Overview

All 37+ backend files organised by folder. Every file has a single clear responsibility.

3.1 Root Files

File	Purpose
server.js	Express app init, middleware mounting, route registration, listen on PORT
package.json	All npm dependencies, dev dependencies, and npm scripts
.env.example	Template for all required environment variables (safe to commit)
.gitignore	Excludes node_modules, .env, uploads/*, coverage/
README.md	Project overview, quick start, API table, documentation index

3.2 config/

File	Purpose
db.js	Mongoose connection to MongoDB, logs host on success, exits on failure
multer.js	Disk storage config, JPEG/PNG/WEBP validation, 5MB size limit

3.3 models/

File	Key Fields & Features
User.js	shopName, email, password (bcrypt pre-save), phone, darkMode (Innovation)
Product.js	name, price, quantity, expiryDate (Innovation), isExpired virtual, isLowStock virtual
Bill.js	items[], subtotal, tax, discount, totalAmount, auto-generated billNumber pre-save

3.4 routes/ & controllers/

Pair	Responsibilities
auth.routes + auth.controller	register, login, getMe, toggleDarkMode — JWT token returned on login
product.routes + product.controller	Full CRUD, search, category filter, low-stock filter, expiry filter
bill.routes + bill.controller	Create draft, finalize (triggers inventory sync), cancel, list with pagination
ocr.routes + ocr.controller	Receives image upload, runs preprocessing + OCR, returns parsed data
pdf.routes + pdf.controller	Generates and streams PDF invoice file for a finalized bill
dashboard.routes + dashboard.controller	Revenue aggregation, top products, today summary with expiry alerts

3.5 services/

File	Description
ocr.service.js	Runs Tesseract OCR on preprocessed image, calls ocrParser, returns structured data
imageProcessor.service.js	Sharp pipeline: grayscale → normalize → sharpen → threshold binarize
inventorySync.service.js	MongoDB bulkWrite to atomically deduct stock on bill finalize
pdfGenerator.service.js	PDFKit invoice with header, item table, totals, footer — streams to response
whatsapp.service.js	Twilio API — sends formatted WhatsApp invoice message to customer [Innovation]

3.6 utils/ & middleware/

File	Purpose
utils/responseHelper.js	successResponse() and errorResponse() — consistent JSON format across all routes
utils/tokenHelper.js	generateToken(userId) and verifyToken(token) wrappers for jsonwebtoken
utils/dateHelper.js	formatDate(), formatCurrency(), getDateRange() for invoice and analytics
utils/ocrParser.js	Regex-based line parser: detects shop name, date formats, and item rows
middleware/auth.middleware.js	Extracts Bearer token, verifies JWT, attaches req.user to request
middleware/error.middleware.js	Global handler for Mongoose validation, duplicate key, and JWT errors

4. Desktop Setup Guide

Follow these steps to run the backend on your local machine (Windows / macOS / Linux).

4.1 Prerequisites

Tool	Version Required	Download
Node.js	v18 or higher	https://nodejs.org
npm	v9 or higher	Included with Node.js
MongoDB	v6 or higher	https://www.mongodb.com/try/download/community
Git	Latest	https://git-scm.com
Postman (optional)	Latest	https://www.postman.com

Step 1 — Verify Node.js

```
node --version    # Must show v18.x.x or higher
npm --version     # Must show v9.x.x or higher
```

Step 2 — Start MongoDB

Windows

```
mongosh    # If connected, MongoDB is running
```

macOS

```
brew services start mongodb/brew/mongodb-community
```

Linux

```
sudo systemctl start mongod
```

Step 3 — Clone & Install

```
git clone https://github.com/YOUR_USERNAME/ai-billing-management.git
cd ai-billing-management
npm install
```

Step 4 — Configure .env

```
cp .env.example .env

# Edit .env and set:
PORT=5000
MONGO_URI=mongodb://localhost:27017/billing_db
JWT_SECRET=any_random_string_here
JWT_EXPIRES_IN=7d

# For WhatsApp feature (optional):
TWILIO_ACCOUNT_SID=your_sid
```

```
TWILIO_AUTH_TOKEN=your_token
TWILIO_WHATSAPP_FROM=whatsapp:+14155238886
```

Step 5 — Run the Server

```
npm run dev    # Development (auto-restarts on save)
npm start      # Production
```

Expected output: "Server running on port 5000" and "MongoDB Connected: localhost"

Step 6 — Test the API

```
# Health check
GET http://localhost:5000/
→ { "message": "AI Billing Management API is running" }

# Register
POST http://localhost:5000/api/auth/register
→ Returns token – use in Authorization header for all other requests
```

Step 7 — Run Tests

```
npm test    # Runs all 12 Jest tests with coverage report
```

4.2 Common Issues & Fixes

Problem	Cause	Fix
MongoDB connection error	MongoDB not running	Start MongoDB (Step 2)
Cannot find module sharp	Native build failed	Run: npm rebuild sharp
Port 5000 already in use	Another app on port	Set PORT=5001 in .env
JWT malformed error	Expired/wrong token	Re-login to get fresh token

5. Special / Innovation Features

Three bonus features were implemented as part of the Intern Innovation Zone, going beyond the core project requirements. All three are fully integrated into the main codebase.

#	Feature	Key File	Route / Trigger
1	 Dark Mode Toggle	controllers/auth.controller.js	PUT /api/auth/darkmode
2	 WhatsApp Invoice Sender	services/whatsapp.service.js	After bill finalize
3	 Expiry Date Tracker	models/Product.js	GET /api/products?expired=true

Feature 1 — Dark Mode Toggle

Type: User Preference / UX

Files: models/User.js | controllers/auth.controller.js | routes/auth.routes.js

Each shop owner's dark mode preference is saved as a boolean in their MongoDB profile. On every login the frontend receives the darkMode value and sets the UI theme automatically — no re-enabling needed per session.

API Usage

```
PUT /api/auth/darkmode
Authorization: Bearer <token>

Response: { "data": { "darkMode": true } }
```

Feature 2 — WhatsApp Invoice Sender

Type: Communication / Third-Party API Integration

Files: services/whatsapp.service.js | innovation/whatsapp.demo.js

After a bill is finalized, the shop owner can send a fully formatted WhatsApp message to the customer's phone using the Twilio API. The customer receives the invoice instantly — no extra app installation required.

Message Includes

- Shop name and bill number
- Full itemized product list with quantities and unit prices
- Subtotal, tax amount, discount applied, and grand total
- Payment method (cash / UPI / card) and thank-you message

Test the Feature

```
node innovation/whatsapp.demo.js
```

Feature 3 — Expiry Date Tracker

Type: Inventory Intelligence / Smart Alerts

Files: models/Product.js | controllers/product.controller.js | controllers/dashboard.controller.js

Every product can store an optional expiryDate. Mongoose virtual properties automatically compute isExpired and isLowStock — no manual checks needed. The dashboard surfaces expiring-soon counts and the API supports an expired filter.

What it Detects Automatically

- isExpired — true if today is past the product's expiry date
- expiringSoonCount — products expiring within the next 30 days (in dashboard summary)
- GET /api/products?expired=true — filter to list only expired stock

Test the Feature

```
node innovation/expiryTracker.demo.js
```

6. Day-by-Day Work Log

Period: Monday 11 May 2026 — Tuesday 26 May 2026 | Sundays excluded

Week 1 — Foundation & Setup

Monday, 11 May 2026 — Project Setup

- Received project brief; studied full requirements document
- Installed Node.js v18, MongoDB, VS Code, Postman
- Initialized project with npm init; installed Express, Mongoose, dotenv, cors
- Created base folder structure: config/, models/, routes/, controllers/, middleware/
- Created server.js with Express init and health-check route
- Created config/db.js for MongoDB connection
- Set up .env.example, .gitignore; pushed first commit to GitHub

Files Created: server.js, config/db.js, package.json, .env.example, .gitignore

Tuesday, 12 May 2026 — Database Models

- Studied Mongoose schema design best practices
- Implemented User.js with bcrypt password hashing pre-save hook
- Implemented Product.js with virtual properties (isLowStock, isExpired)
- Implemented Bill.js with nested BillItemSchema and auto-generated bill numbers
- Added owner-scoped indexes for efficient queries
- Verified schemas using mongosh

Files Created: models/User.js, models/Product.js, models/Bill.js

Wednesday, 13 May 2026 — Authentication System

- Implemented JWT-based stateless authentication
- Created utils/tokenHelper.js for generate/verify JWT
- Created utils/responseHelper.js for standardized success/error format
- Implemented auth.controller.js: register, login, getMe, toggleDarkMode
- Created auth.middleware.js: extracts and verifies Bearer tokens
- Created auth.routes.js and mounted on server
- Tested all auth endpoints in Postman — all working

Files Created: auth.controller.js, auth.middleware.js, auth.routes.js, tokenHelper.js, responseHelper.js

Thursday, 14 May 2026 — Product / Inventory API

- Implemented full CRUD for products in product.controller.js
- Added search by name, filter by category, low-stock filter, expired filter (Innovation)
- Implemented soft-delete (isActive: false) to preserve bill history integrity
- Created product.routes.js with /categories endpoint
- Tested all 6 product endpoints in Postman

Files Created: controllers/product.controller.js, routes/product.routes.js

Friday, 15 May 2026 — Billing API & Inventory Sync

- Implemented `bill.controller.js`: `createBill`, `finalizeBill`, `getBills`, `getBill`, `cancelBill`
- Added automatic subtotal/tax/discount/total calculation on bill creation
- Implemented `inventorySync.service.js` using MongoDB `bulkWrite` for atomic stock deduction
- Connected inventory sync to `PUT /bills/:id/finalize`
- Added pagination (page, limit) and date-range filtering on bill list
- Tested complete billing flow end-to-end

Files Created: `bill.controller.js`, `bill.routes.js`, `inventorySync.service.js`

Week 2 — AI Features & PDF

Monday, 18 May 2026 — Image Preprocessing

- Researched OCR accuracy improvement techniques
- Installed Sharp library and studied pipeline API
- Implemented `imageProcessor.service.js`: `grayscale` → `normalize` → `sharpen` → `binarize`
- Configured `config/multer.js`: JPEG/PNG/WEBP only, 5MB limit, unique filenames
- Tested preprocessing on real bill photos; OCR accuracy improved significantly

Files Created: `services/imageProcessor.service.js`, `config/multer.js`

Tuesday, 19 May 2026 — OCR Integration

- Installed and configured Tesseract.js
- Implemented `services/ocr.service.js` for text extraction
- Implemented `utils/ocrParser.js`: regex to detect shop name, date, item rows
- Created `ocr.controller.js` and `ocr.routes.js`
- Tested OCR on 5 different real bill images — achieved ~80% accuracy
- Added automatic cleanup of temp preprocessed images

Files Created: `ocr.service.js`, `ocrParser.js`, `ocr.controller.js`, `ocr.routes.js`

Wednesday, 20 May 2026 — PDF Invoice Generation

- Studied PDFKit API and professional invoice layout design
- Implemented `pdfGenerator.service.js` with full layout:
 - Shop header: name, address, phone, email
 - Invoice metadata: bill number, date, customer, payment method
 - Items table with alternating row colors
 - Totals block: subtotal, tax, discount, grand total
 - Professional footer with thank-you message
- Streamed PDF directly to Express response (no temp file saved)

Files Created: `pdfGenerator.service.js`, `pdf.controller.js`, `pdf.routes.js`

Thursday, 21 May 2026 — Dashboard Analytics

- Implemented `dashboard.controller.js` with three endpoints:
 - Summary: today revenue, total bills, total products, `lowStockCount`, `expiringSoonCount`
 - Daily revenue: MongoDB \$group aggregation by date (for bar chart)

- → Top products: \$unwind + \$group on bill items by quantity
- Created utils/dateHelper.js with getDateRange(), formatDate(), formatCurrency()
- Tested all aggregation pipelines with real data

Files Created: dashboard.controller.js, dashboard.routes.js, dateHelper.js

Friday, 22 May 2026 — Innovation Zone Features

- Feature 1 — Dark Mode: added darkMode field to User, toggleDarkMode controller, PUT route
- Feature 2 — WhatsApp Sender: Twilio SDK, whatsapp.service.js, formatted message template
- Feature 3 — Expiry Tracker: expiryDate field, isExpired virtual, expiringSoonCount in dashboard
- Created innovation/ folder with README.md and two demo scripts
- Tested all three features; added Twilio keys to .env.example

Files Created: whatsapp.service.js, innovation/README.md, whatsapp.demo.js, expiryTracker.demo.js

Week 3 — Testing, Documentation & Final Push

Monday, 25 May 2026 — Tests & Documentation

- Wrote tests/auth.test.js: register, login, duplicate email, wrong password (4 tests)
- Wrote tests/bill.test.js: create draft, fetch list, finalize bill (3 tests)
- Wrote tests/ocr.test.js: 4 unit tests for ocrParser.js
- All 12 tests passing — npm test output: 12 passed, 0 failed
- Created middleware/error.middleware.js for global error handling
- Wrote docs/ARCHITECTURE.md — layer diagram, tech stack, data flow
- Wrote docs/API_REFERENCE.md — all 19 endpoints with request/response examples
- Wrote docs/SETUP_GUIDE.md — step-by-step for Windows/macOS/Linux
- Wrote docs/SPECIAL_FEATURES.md and docs/WORK_LOG.md

Files Created: All test files, error.middleware.js, complete docs/ folder

Tuesday, 26 May 2026 — Final Review & GitHub Submission

- Reviewed all 37+ code files for consistency and clean-up
- Verified .gitignore excludes .env, node_modules/, uploads/*
- Updated root README.md with full project overview and documentation index
- Ran npm test — 12/12 tests passing
- Ran npm run dev — all 19 API endpoints responding correctly
- Created comprehensive project report (this document)
- Final commit and push to GitHub — project complete

6.1 Work Summary

Week	Days	Focus	Key Deliverables
Week 1	Mon 11 – Fri 15 May	Foundation	Setup, models, auth, products, billing + inventory sync

Week 2	Mon 18 – Fri 22 May	AI & Features	OCR, image processing, PDF, dashboard, innovation zone
Week 3	Mon 25 – Tue 26 May	Polish & Docs	Tests, documentation, final GitHub push

Metric	Count
Total Files Created	37+
API Endpoints	19
Tests Written	12 (all passing)
Innovation Features	3
Working Days	12